

Informe Técnico: Análisis del Modelo UML — Sistema EcoTech Solutions

Curso: Programación Orientada a Objetos Seguro (TI3V21)
Asignatura/Actividad: Diseño de sistema para EcoTech Solutions
Tipo de documento: Informe técnico de evaluación del diagrama de clases UML

1. Introducción

El presente informe analiza el diagrama de clases UML diseñado para el sistema de gestión de EcoTech Solutions, evaluando su correspondencia con los requerimientos del problema modelado y su alineación con los criterios de la rúbrica de evaluación establecidos para la actividad. El diagrama incluye siete clases (Departamento, Usuario, Empleado, Administrador, ReporteManager, Proyecto, RegistroTiempo) relacionadas mediante herencia, agregación y asociaciones con multiplicidad definida.

2. Descripción general del modelo

Clase	Rol en el sistema	Atributos clave	Métodos clave
Usuario (abstracta)	Superclase base para todo actor del sistema	id, nombre, correo	puede_acceder(mo
Empleado	Hereda de Usuario; representa al trabajador	direccion, telefono, salario, fecha_inicio, id_departamento	get_salario(), set_sa registrar_horas(), obtener_proyectos()
Administrador	Hereda de Usuario; gestiona la organización	—	crear_departament generar_informe()
Departamento	Agrupar empleados	id_dept, nombre_dept, gerente	agregar_empleado(remover_empleado listar_empleados()
Proyecto	Unidad de trabajo asignable a empleados	id_proyecto, nombre, descripcion, fecha_inicio, estado	asignar_empleado(remover_empleado calcular_total_hora
RegistroTiempo	Registra horas trabajadas por proyecto	id_registro, fecha, horas, descripcion	validar_limite_diari obtener_resumen()
ReporteManager	Genera reportes exportables	id_reporte, tipo, fecha_generacion	generar_reporte_er exportar_pdf(), exportar_excel()

Relaciones identificadas: - **Herencia:** Empleado y Administrador heredan de Usuario (abstracta). - **Agregación:** Departamento (1) ◇— Empleado (0..*) — un departamento agrupa empleados sin poseerlos en ciclo de vida. - **Asociación “Asignado”:** Empleado (0..*) — Proyecto (0..*), relación muchos a muchos. - **Asociación “Registra”:** Empleado (1) — RegistroTiempo (0..*). - **Asociación “Pertenece”:** RegistroTiempo (0..*) — Proyecto (1). - **Dependencias (líneas punteadas):** Administrador y ReporteManager dependen de Empleado/Proyecto para operar (uso de instancias como parámetro o fuente de datos), sin relación estructural permanente.

3. Análisis según criterios de la rúbrica

3.1 Diseño de clases y correspondencia modelo-código (2.1.1.G.1 / 2.1.1.I.2)

El modelo define correctamente atributos, visibilidad (- privado, + público) y métodos por clase, alineados con un dominio de recursos humanos y gestión de proyectos. Las relaciones de herencia (Usuario → Empleado, Usuario → Administrador) y de agregación (Departamento → Empleado) están explícitamente representadas con la notación UML correspondiente (triángulo hueco para herencia, rombo hueco para agregación), lo que facilita su traducción directa a código Python: las subclases heredarían atributos y el método abstracto `puede_acceder()`, y Departamento mantendría una colección de referencias a Empleado sin gestionar su ciclo de vida.

Un punto a reforzar de cara a la implementación: el diagrama no explicita si `puede_acceder(String modulo): bool` es un método abstracto puro o si tiene una implementación por defecto, lo cual debe justificarse verbalmente al construir el código (uso de `ABC/@abstractmethod` en Python).

3.2 Organización, encapsulamiento y abstracción (2.1.2.G.3 / 2.1.2.I.4)

El encapsulamiento está correctamente aplicado: todos los atributos son privados (-) y el acceso se realiza mediante métodos públicos (por ejemplo, `get_salario()` / `set_salario()` en Empleado). La abstracción se resuelve mediante la clase «abstract» Usuario, que evita duplicar los atributos `id`, `nombre` y `correo` en Empleado y Administrador, reduciendo redundancia.

Como oportunidad de mejora: Administrador no declara atributos propios, lo que es válido si su rol es puramente funcional, pero conviene justificar explícitamente esta decisión (evitar una subclase “vacía” sin argumento de diseño) para sostener el criterio de justificación arquitectónica (I.4).

3.3 Conexión con base de datos (2.1.3.G.5 / 2.1.3.I.6)

No aplica en esta etapa. El diagrama entregado corresponde al modelo estático de clases (diseño) y no incluye una capa de persistencia ni referencias a una librería de conexión a base de datos (por ejemplo, `sqlite3`, `psycopg2`, `mysql-connector`). Este criterio deberá evaluarse en la fase de implementación del código, cuando se defina cómo cada clase (especialmente Empleado, Proyecto y RegistroTiempo) se persiste y se ejecutan operaciones CRUD.

3.4 Manejo de excepciones (2.1.4.G.7 / 2.1.4.I.8)

No aplica en esta etapa. El diagrama de clases no modela explícitamente el manejo de errores (esto es esperado, ya que UML de clases no suele representar bloques try/except). Se recomienda que, durante la codificación, métodos sensibles como `set_salario()`, `validar_limite_diario()` y las operaciones de `ReporteManager` (`exportar_pdf`, `exportar_excel`) incorporen validación de entradas y manejo de excepciones, dado que dependen de datos externos (rutas de archivo, valores numéricos).

3.5 Evaluación crítica del apoyo de herramientas de IA (2.1.5.G.9 / 2.1.5.I.10)

En la construcción de este diagrama es importante documentar explícitamente qué elementos fueron sugeridos por una herramienta de IA (por ejemplo, la estructura general de clases o los nombres de métodos) y cuáles fueron decisiones propias. Del análisis del modelo se identifican dos observaciones que un proceso de revisión crítica debería registrar:

1. **Ambigüedad en las dependencias punteadas:** las relaciones de dependencia entre `Administrador/ReporteManager` y `Empleado/Proyecto` no tienen una etiqueta ni multiplicidad explícita, lo que dificulta interpretar si `ReporteManager.generar_reporte_empleado()` también debería depender de `Proyecto` (dado que un empleado puede tener horas asociadas a varios proyectos). Esto sugiere revisar si fue una sugerencia automática que requiere ajuste manual.
2. **Rol vacío de Administrador:** la ausencia de atributos propios podría ser resultado de una generación genérica; se recomienda evaluar si el rol necesita atributos adicionales (por ejemplo, nivel de permisos) antes de aceptar el diseño tal como está.

Documentar este tipo de hallazgos —con criterios técnicos de coherencia y suficiencia del modelo— es lo que sustenta el cumplimiento de este criterio, más allá de simplemente declarar que se usó una IA.

4. Observaciones y recomendaciones generales

- Formalizar la multiplicidad y el nombre de las relaciones de dependencia (actualmente sin etiqueta) para evitar ambigüedad en la fase de codificación.
- Justificar por qué `Administrador` no requiere atributos propios, o incorporar los que correspondan según el caso de uso real.
- Preparar, en el informe final, una sección específica que documente qué partes del diagrama fueron generadas/sugeridas por IA y qué criterios se usaron para aceptarlas o modificarlas (trazabilidad exigida por el criterio 2.1.5).

5. Conclusión

El modelo UML de EcoTech Solutions cumple con los principios fundamentales de diseño orientado a objetos: encapsulamiento, herencia y abstracción están correctamente aplicados, y la correspondencia entre el diagrama y una futura implementación en Python es clara. Los criterios relacionados con conexión a base de datos y manejo de excepciones corresponden a una etapa posterior del desarrollo y deben abordarse cuando se traduzca el diseño a

código funcional. Finalmente, se identifican dos puntos de ambigüedad en el diagrama que deberían resolverse y documentarse como parte del ejercicio de evaluación crítica exigido por la rúbrica.